React Testing Practice Questions

Vitest + React Testing Library | Beginner to Advanced

Section 1:

Question 1: Greet Component

Component to test: Greet.jsx

Task: Write tests for the following scenarios:

Test Case	Expected Result
Renders "Hello world"	Text should appear (case-insensitive)
Does NOT render "Goodbye"	"Goodbye" should not be in the document

Hint: Use screen.getByText() with { exact: false } for case-insensitive matching.

Question 2: AddPeriod Component

Component to test: AddPeriod.jsx

Task: Write tests for the following scenarios:

Test Case	Code Example
No name prop provided	<AddPeriod />
Name prop provided	<AddPeriod name="John" />
Empty string as name	<AddPeriod name="" />

Hint: The component adds a period at the end of the greeting.

Section 2:

Question 3: Application Component

Component to test: Application.jsx

Write tests for the following elements:

Element	Test Requirement
Name input field	Verify value="Alan" and placeholder text
Job Location dropdown	Verify number of options (should be 4 including default)
Bio textarea	Verify it exists in the document
Terms checkbox	Verify it exists and is NOT checked by default
Submit button	Verify it exists using getByRole("button")
h1 heading	Verify text "Job Application Form"
h2 heading	Verify text "For internal applicants"

Bonus: Test that the Terms checkbox can be checked/unchecked using userEvent.click().

Question 4: Counter Component - Basic Interactions

Component to test: Counter.jsx

Test Case	Steps
Initial render	Render component
Increment button	Click Increment once
Increment multiple times	Click Increment 3 times

Test Case	Steps
Set button with valid number	Type "5" in input, click Set
Set button with zero	Type "0", click Set

Hint: Use userEvent.type() for input and userEvent.click() for buttons.

Section 3: Async & Complex Behavior

Question 5: Skills Component - Async Rendering

Component to test: Skills.jsx

Component behavior:

- Initially displays "Log In" button
- After 1500ms, automatically shows "Start Learning" button
- Clicking "Log In" immediately shows "Start Learning"

Test Case	Method to Use	Expected Result
Initial render	getByRole	"Log In" button present
Start Learning not present initially	queryByRole	"Start Learning" NOT in document
Start Learning appears after timeout	findByRole	Button appears within 2000ms
Click Log In shows Start Learning	userEvent.click() + findByRole	Button appears immediately

Bonus: Use vi.useFakeTimers() to speed up the 1500ms timeout.

Question 6: Skills Component - List Rendering

Given skills array: ["HTML", "CSS", "JavaScript"]

Test Case	Verification
Unordered list exists	getByRole("list")
Number of list items	Should be 3
Each skill text appears	"HTML", "CSS", "JavaScript" all present

Section 4: Advanced / Edge Cases

Question 7: Counter Component - Edge Cases

Edge Case	Steps	Expected Behavio
Non-numeric input	Type "abc" in amount input	Input should show
Empty input	Type nothing, click Set	Count should rema
Negative number	Type "-5", click Set	Count should show
Decimal number	Type "3.7", click Set	parseInt behavior -
Very large number	Type "999999999", click Set	Should handle with
Set then Increment	Set to 10, then Increment	Count becomes 11

Challenge: Fix the existing test that expects "20" after typing "200" - why does it fail?

Question 8: Application Component - Form Fields Accessibility

Test accessibility requirements:

Field	Accessibility Check
Name input	Has associated <label> with htmlFor="name"
Job Location	Has associated label (currently missing - identify the issue)
Bio	Has associated <label> with htmlFor="bio"
Terms	Has associated <label> with htmlFor="terms"

Task: Identify which field has an accessibility issue and write a test that would fail until fixed.

Question 9: Counter Component - Tab Order (Fix Broken Test)

Current broken test in Counter.test.jsx:

```
test("elements are focused in the right order", async () => {
  const amountInput = screen.getByRole("spinbutton");
  const setButton = screen.getByRole("button", { name: "Set" });
  const incrementButton = screen.getByRole("button", { name: "Increment" });

  await user.tab();
  await user.tab();
  expect(incrementButton).toHaveFocus();
  await user.tab();
  expect(amountInput).toHaveFocus();
  await user.tab();
  expect(setButton).toHaveFocus();
});
```

Tasks:

1. Run the test - it will fail
2. Identify why the tab order is different from expected
3. Rewrite the test to match the actual tab order
4. (Optional) Modify the component to change tab order using tabIndex

Answer Template

For each test, use this structure:

```
import { test, expect, describe } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import "@testing-library/jest-dom/vitest";
import ComponentName from "./ComponentName";

describe("ComponentName", () => {
  test("test description", () => {
    // Arrange
    render(<ComponentName />);

    // Act (if needed)

    // Assert
    expect(element).toBeInTheDocument();
  });
});
```

Quick Reference: Common Queries

Element Type	Query Method
Button	getByRole("button")
Text input	getByRole("textbox")
Number input	getByRole("spinbutton")
Checkbox	getByRole("checkbox")
Dropdown	getByRole("combobox")
Heading	getByRole("heading", { level: 1 })
List	getByRole("list")
List item	getByRole("listitem")
By label text	getByLabelText("Label Text")
By display value	getByDisplayValue("value")
By test ID	getByTestId("test-id")

Submission Checklist

- ☐ All tests pass
- ☐ Async tests use findBy or waitFor
- ☐ User interactions use userEvent
- ☐ Each test is independent (no shared state)